École Normale Supérieure de l'Enseignement Technique de Mohammedia
Université Hassan II de Casablanca

RAPPORT DE L'ÉPREUVE TECHNIQUE

Architecture microservices, IA et développement full-stack

TEST ADRIA ENSET 2027

Réalisé par	Wahyudi Muhammad Irfan
Repository	Wahyudi-Muhammadirfan-test-Adria
Année	2026-2027

Spring Boot - Spring Cloud - Spring AI - Kafka - Angular - Python ML - Docker

Sommaire

Section	Contenu	Page
1	Introduction et synthèse	3
2	Partie A - Préparation et organisation	4
3	Partie B - Architecture générale	5
4	Partie B - Diagramme de classes	6
5	Partie C - Microservices métier	7
6	Parties C et D - Connecteurs et services techniques	8
7	Parties E et F - Agent IA et Kafka	9
8	Partie G - Client Angular	10
9	Partie H - Détection de fraude	11-12
10	Partie I - DevOps	13-14
11	Validation et conclusion	15

1. Introduction et synthèse

L'objectif est de réaliser une plateforme bancaire modulaire permettant de consulter des comptes, d'enregistrer des transactions, de publier des événements et d'interroger les données avec un assistant conversationnel. La solution couvre également un client Angular, une expérimentation de détection de fraude et une chaîne DevOps conteneurisée.

1.1 Objectifs techniques

- Séparer les responsabilités dans des microservices Spring Boot 4.1.2 et Java 21.
- Exposer les mêmes données via REST, GraphQL et des outils MCP.
- Sécuriser les règles métier de création d'une transaction et tolérer les pannes distantes.
- Publier les événements de transaction dans Kafka et les consommer sans modifier les données.
- Proposer un dashboard Angular et un pipeline Python de classification de fraude.
- Préparer les images Docker, Docker Compose et un pipeline GitHub Actions.

1.2 Livrables réalisés

Domaine	Réalisation
Métier	account-service et transaction-service avec H2, validations et données de démonstration
Spring Cloud	Gateway, Config Server, Eureka, OpenFeign et Resilience4j
Connecteurs	REST, GraphQL et MCP partageant la couche service
IA et événements	Agent Spring AI, producteur Kafka et notification-service consumer
Interface	Application Angular pour comptes, transactions et création d'opérations
Data science	Pipeline scikit-learn reproductible et résultats exportés
DevOps	Dockerfiles, Compose, GitHub Actions et stratégie d'exploitation

2. Partie A - Préparation et organisation

Le projet est versionné dans un repository GitHub dédié. Les fonctionnalités ont été développées et validées par étapes avec des commits séparés, ce qui facilite la relecture et le retour arrière. Chaque microservice possède son propre fichier Maven et son wrapper Maven.

2.1 Organisation du repository

```
test-Adria/
|-- account-service/         REST, GraphQL, MCP, H2
|-- transaction-service/     Feign, Resilience4j, Kafka
|-- gateway-service/         Spring Cloud Gateway
|-- discovery-service/       Eureka Server
|-- config-service/          Spring Cloud Config
|-- ai-service/              Spring AI et client MCP
|-- notification-service/    Consumer Kafka
|-- frontend/                Angular
|-- fraud-detection/         Python / scikit-learn
|-- docker-compose.yml
`-- .github/workflows/ci-cd.yml
```

2.2 Jalons de développement

Jalon	Résultat
Services métier	Entités, repositories, services et API REST
Communication	OpenFeign, contrôles métier et débit du compte
Cloud	Gateway, Eureka et configuration centralisée
Connecteurs	GraphQL, MCP et chatbot bancaire
Événements	Publication et consommation Kafka
Interfaces	Dashboard Angular et formulaire de transaction
IA Python	Dataset synthétique, modèle et évaluation
DevOps	Conteneurs, Compose et pipeline CI/CD

Les opérations externes de soumission (Google Sheet, visibilité finale du repository et dépôt Classroom) restent des actions manuelles du candidat.

3. Partie B - Architecture générale

Le client Angular traverse la Gateway pour accéder aux services métier. Le service de transactions consulte account-service avec OpenFeign, puis publie un événement Kafka après une persistance réussie. L'agent Spring AI utilise les outils MCP exposés par les services métier.

Architecture de la plateforme bancaire

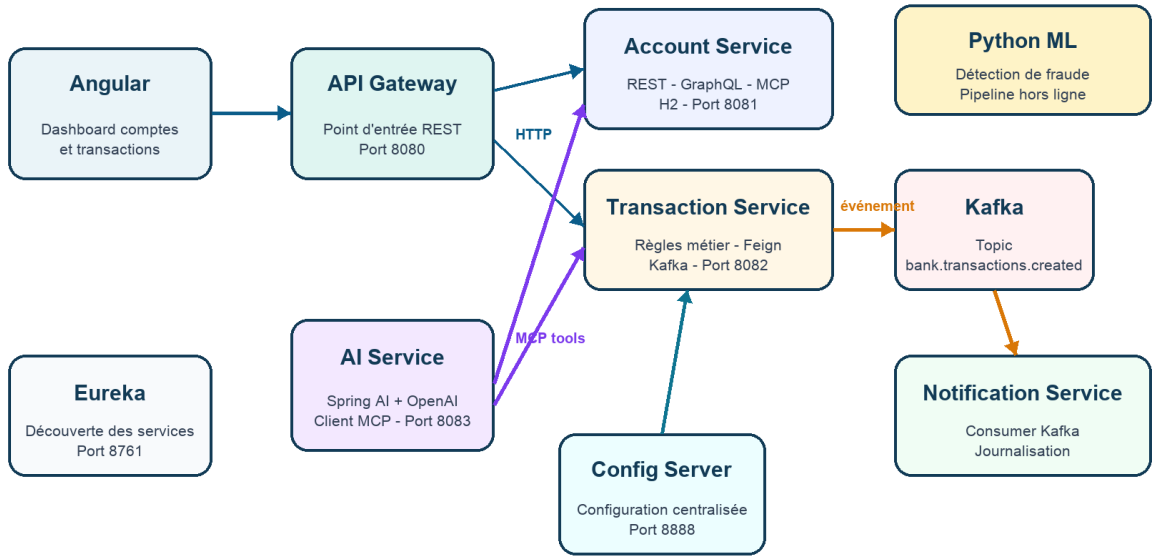


Figure 1 - Schéma d'architecture de la plateforme.

Choix structurants

- Les contrôleurs REST, GraphQL et MCP délèguent aux mêmes services métier.
- Les échanges synchrones utilisent HTTP ; Kafka porte les notifications asynchrones.
- Le modèle Python reste indépendant : son intégration aux microservices n'est pas demandée.

4. Partie B - Modèle de classes

Account représente l'état financier d'un client. BankTransaction conserve l'identifiant du compte plutôt qu'une relation JPA distante, car chaque microservice possède son propre modèle et sa propre base.

Diagramme de classes métier

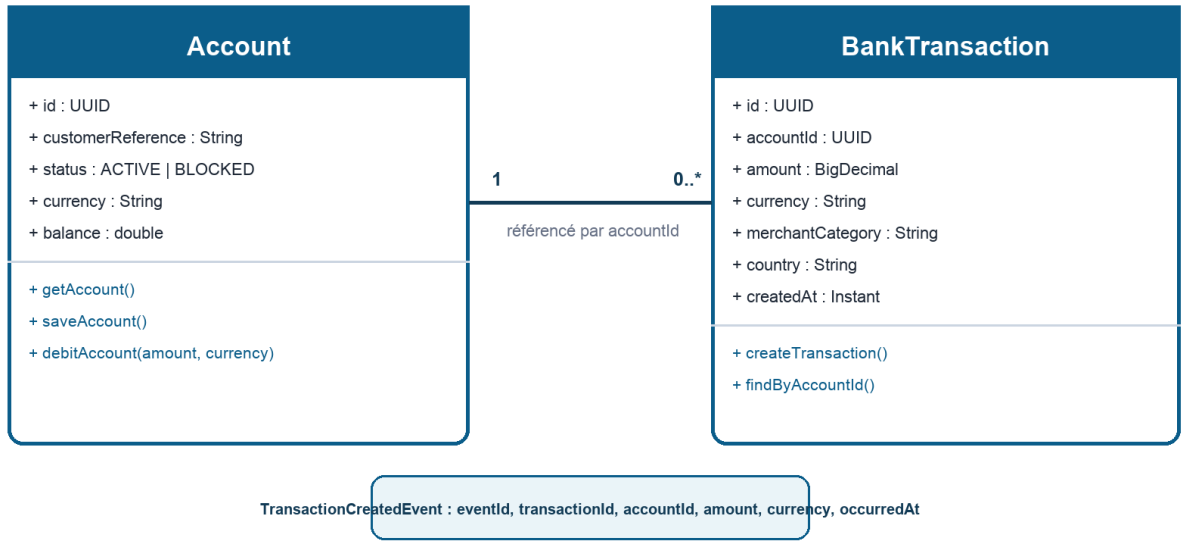


Figure 2 - Diagramme de classes métier simplifié.

Règles d'intégrité

- Le montant doit être strictement positif et inférieur ou égal au plafond configuré (1000).
- Le compte doit exister, être ACTIVE, utiliser la même devise et disposer d'un solde suffisant.
- Le débit est protégé localement par une transaction et un verrou pessimiste dans account-service.
- La transaction bancaire est persistée puis l'événement TransactionCreated est publié.

Limite distribuée : le débit distant et la persistance de la transaction ne forment pas une transaction atomique globale. Une stratégie Saga ou Outbox serait nécessaire pour une garantie de production.

5. Partie C - Microservices métier

5.1 Account Service

Ce service gère les comptes, leur statut, leur devise et leur solde. Il fournit la consultation, la création et le débit sécurisé d'un compte. Trois comptes de démonstration sont initialisés au démarrage.

Endpoint	Rôle
GET /accounts	Lister les comptes
GET /accounts/{id}	Consulter un compte
POST /accounts	Créer un compte
POST /accounts/{id}/debit	Débiter après validation
POST /graphql	Requêtes GraphQL accounts et account
MCP tools	list_accounts et get_account

5.2 Transaction Service

Le service enregistre et consulte les transactions. POST /transactions contrôle le montant, l'existence et l'état du compte, la devise et le solde. Il déclenche ensuite le débit distant, persiste l'opération et publie l'événement Kafka. Cinq transactions de démonstration sont disponibles.

Endpoint	Rôle
GET /transactions	Lister les transactions
GET /transactions/{id}	Consulter une transaction
GET /transactions/account/{accountId}	Filtrer par compte
POST /transactions	Créer une transaction validée
POST /graphql	Requêtes GraphQL de consultation
MCP tools	list_transactions et list_transactions_by_account

6. Parties C et D - Communication et services techniques

6.1 OpenFeign et Resilience4j

Transaction-service utilise un client OpenFeign pour consulter et débiter account-service. Resilience4j encapsule l'appel de consultation avec un circuit breaker. En cas d'indisponibilité, le fallback retourne une erreur de service claire au lieu de bloquer indéfiniment la requête.

Paramètre	Valeur
Fenêtre glissante	4 appels
Appels minimum	2
Seuil d'échec	50 %
Attente avant semi-ouverture	10 secondes

6.2 Services Spring Cloud

Composant	Port	Fonction
Gateway	8080	Routes /accounts/** et /transactions/**
Eureka	8761	Registre de découverte des services
Config Server	8888	Expose bank.transaction.max-amount=1000
Account	8081	Service métier des comptes
Transaction	8082	Service métier des transactions
AI	8083	Assistant bancaire

6.3 Contrat et erreurs

- 404 lorsqu'un compte ou une transaction n'existe pas.
- 400 pour montant invalide, compte bloqué, devise différente ou solde insuffisant.
- 503 lorsque le service des comptes est indisponible via le mécanisme de fallback.

7. Parties E et F - Agent IA et Kafka

7.1 Chatbot bancaire agentique

AI-service expose POST /chat. Spring AI construit un ChatClient utilisant le modèle gpt-4o-mini et les outils MCP des services Account et Transaction. Le prompt système demande de ne jamais inventer de données bancaires. La clé OPENAI_API_KEY est injectée par variable d'environnement.

```
POST /chat
Content-Type: application/json

{ "message": "Liste les comptes actifs et leur solde" }
```

7.2 Communication asynchrone Kafka

Après la persistance d'une transaction, TransactionEventPublisher envoie un événement dans bank.transactions.created. accountId est utilisé comme clé afin de préserver l'ordre des événements d'un même compte. notification-service consomme et journalise l'événement sans modifier les données bancaires.

```
{
  "eventId": "UUID",
  "eventType": "TransactionCreated",
  "transactionId": "UUID",
  "accountId": "UUID",
  "amount": 250.00,
  "currency": "MAD",
  "occurredAt": "2026-10-30T11:41:01Z"
}
```

- Test unitaire du producteur : vérification du topic, de la clé et du contenu publié.
- Test du consumer : réception et journalisation d'un TransactionCreatedEvent.

8. Partie G - Client web Angular

Le client Angular communique avec les API via la Gateway. Le dashboard affiche les comptes, leur statut et les soldes, puis permet de consulter les transactions et de créer une nouvelle opération. Un proxy de développement redirige les appels vers localhost:8080.

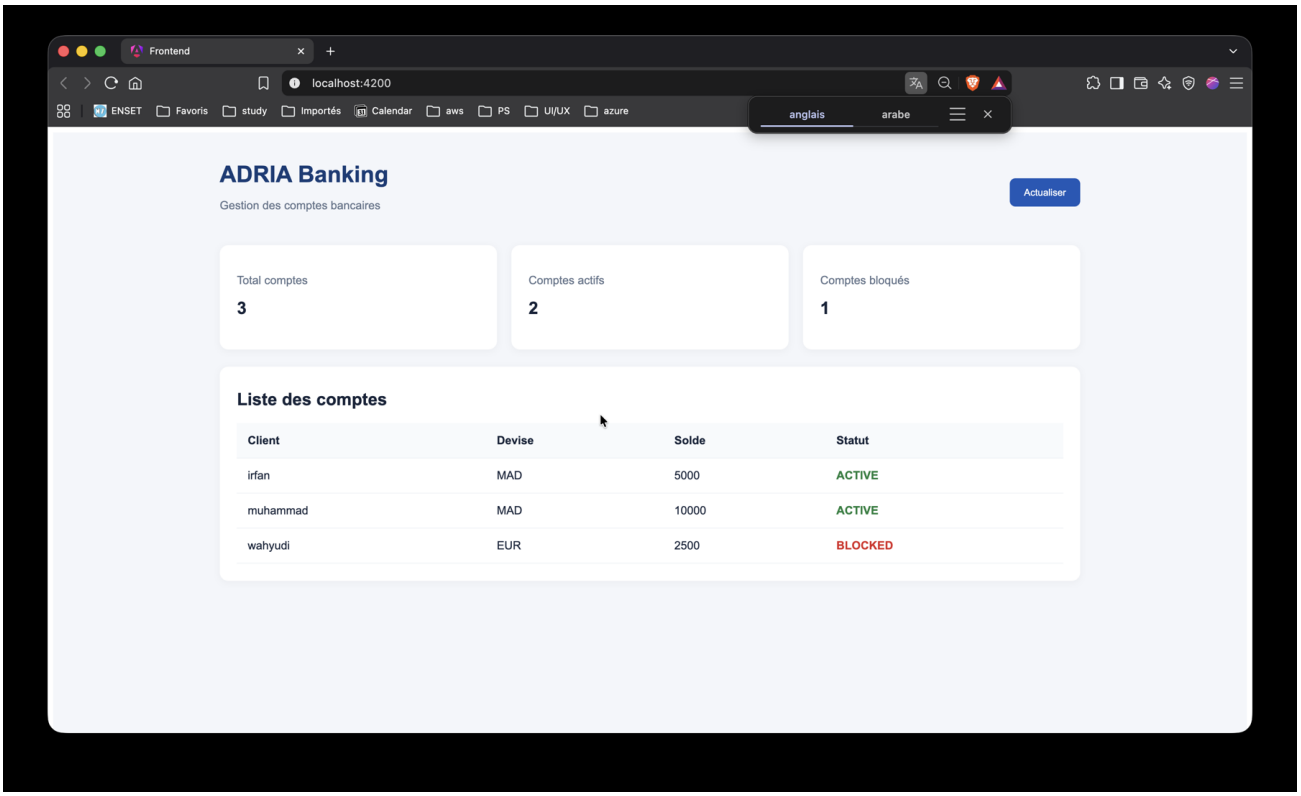


Figure 3 - Dashboard Angular alimenté par les données du backend.

Fonctionnalités validées

- Synthèse du nombre total de comptes, comptes actifs et comptes bloqués.
- Liste des comptes avec client, devise, solde et statut.
- Liste des transactions bancaires et formulaire POST /transactions.
- Build de production Angular réussi : bundle principal de 275,78 kB.

9. Partie H - Détection de fraude : méthode

Un dataset synthétique de 1000 transactions a été construit avec 50 fraudes (5 %). Il contient des valeurs manquantes dans hour, country, merchant_category et average_amount_last_7d afin de tester l'imputation.

9.1 Pipeline sans fuite de données

- Séparation stratifiée : 800 lignes d'entraînement et 200 lignes de test.
- Exclusion des identifiants transaction_id et account_id.
- Imputation médiane des nombres et valeur la plus fréquente pour les catégories.
- One-hot encoding, normalisation et régression logistique avec class_weight=balanced.
- Prétraitement ajusté uniquement sur l'ensemble d'entraînement ; seuil fixé à 0,5.

Mesure	Résultat
Précision fraude	0,600
Rappel fraude	0,900
F1-score fraude	0,720
Average precision	0,936
Matrice de confusion	[[184, 6], [1, 9]]

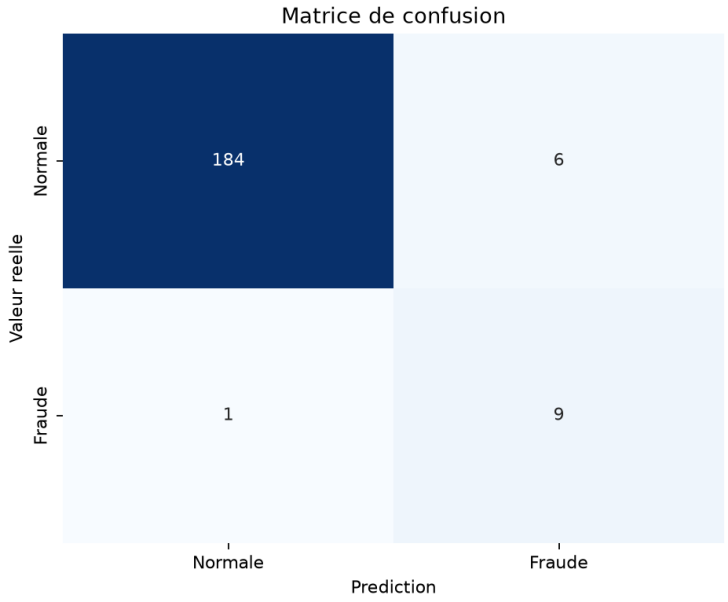


Figure 4 - Matrice de confusion sur les 200 transactions de test.

10. Partie H - Analyse des résultats

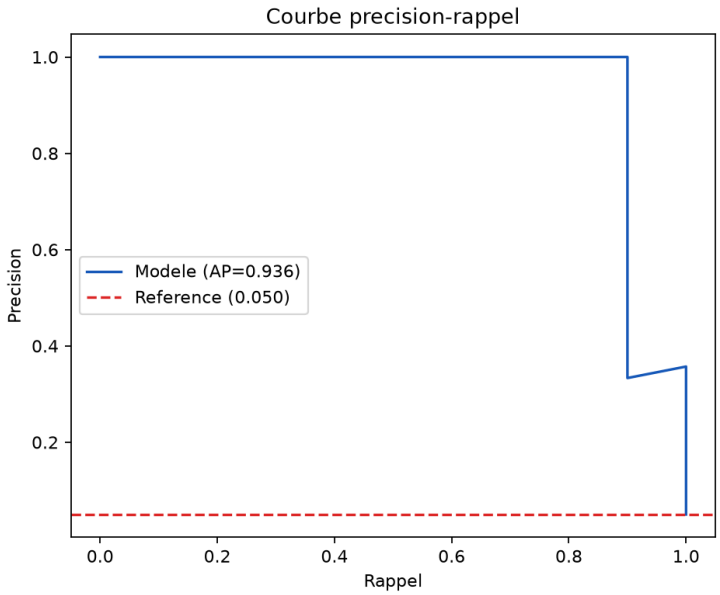


Figure 5 - Courbe précision-rappel, AP=0,936 contre une référence à 0,05.

10.1 Pourquoi l'accuracy seule est-elle insuffisante ?

Avec seulement 5 % de fraudes, un modèle prédisant toujours la classe normale obtiendrait 95 % d'accuracy tout en ratant toutes les fraudes. Il faut donc analyser la précision, le rappel, le F1-score et l'average precision.

10.2 Compromis entre faux positifs et fraudes non détectées

Un rappel élevé détecte plus de fraudes mais augmente les contrôles et blocages injustifiés. Une précision élevée réduit ces alertes, au risque de laisser passer davantage de fraudes. Le seuil dépend des coûts métier.

10.3 Effet d'une diminution du seuil

Davantage de transactions seraient classées frauduleuses : le rappel augmenterait généralement, tandis que la précision pourrait diminuer à cause des faux positifs supplémentaires.

10.4 Intérêt d'une validation temporelle

Les comportements et les techniques de fraude évoluent. Entraîner sur le passé et tester sur une période ultérieure reflète mieux la production et réduit les risques de fuite d'information entre futur et passé.

11. Partie I - Conteneurisation et CI/CD

11.1 Docker et Docker Compose

Chaque service Java possède un Dockerfile multi-stage : compilation avec Eclipse Temurin JDK 21, puis exécution avec un JRE 21 sous un utilisateur non privilégié. Docker Compose orchestre Kafka et les sept services en injectant les URLs internes par variables d'environnement.

Conteneur	Image / build	Port hôte
gateway-service	./gateway-service	8080
account-service	./account-service	8081
transaction-service	./transaction-service	8082
ai-service	./ai-service	8083
config-service	./config-service	8888
discovery-service	./discovery-service	8761
kafka	apache/kafka:4.0.0	9092
notification-service	./notification-service	interne

11.2 Pipeline GitHub Actions

Étape	Mise en œuvre
Compilation et tests	Matrice Maven sur les 7 services avec ./mvnw verify
Construction	docker/build-push-action avec le contexte de chaque service
Publication	GitHub Container Registry, tags latest et SHA Git
Déploiement	Étape schématique dépendante de la publication des images

```
docker compose up --build
# Accès principal : http://localhost:8080
docker compose down
```

12. Partie I - Exploitation et validation

12.1 Gestion des secrets

Les ports, URLs et paramètres métier sont de la configuration versionnable. Les clés, mots de passe et jetons sont des secrets : ils restent hors du repository et sont injectés via .env, GitHub Secrets ou un gestionnaire de secrets.

12.2 Santé et observabilité

Les sondes startup, readiness et liveness distinguent démarrage, disponibilité et vitalité. Les logs sont émis sur stdout ; Micrometer/Prometheus et Grafana couvrent les métriques ; OpenTelemetry relie les traces entre Gateway, services et Kafka.

12.3 Stratégie de rollback

Chaque image possède un tag immuable basé sur le SHA Git. En cas d'incident, le dernier tag stable est redéployé puis contrôlé par les sondes. Les évolutions de données doivent rester rétrocompatibles.

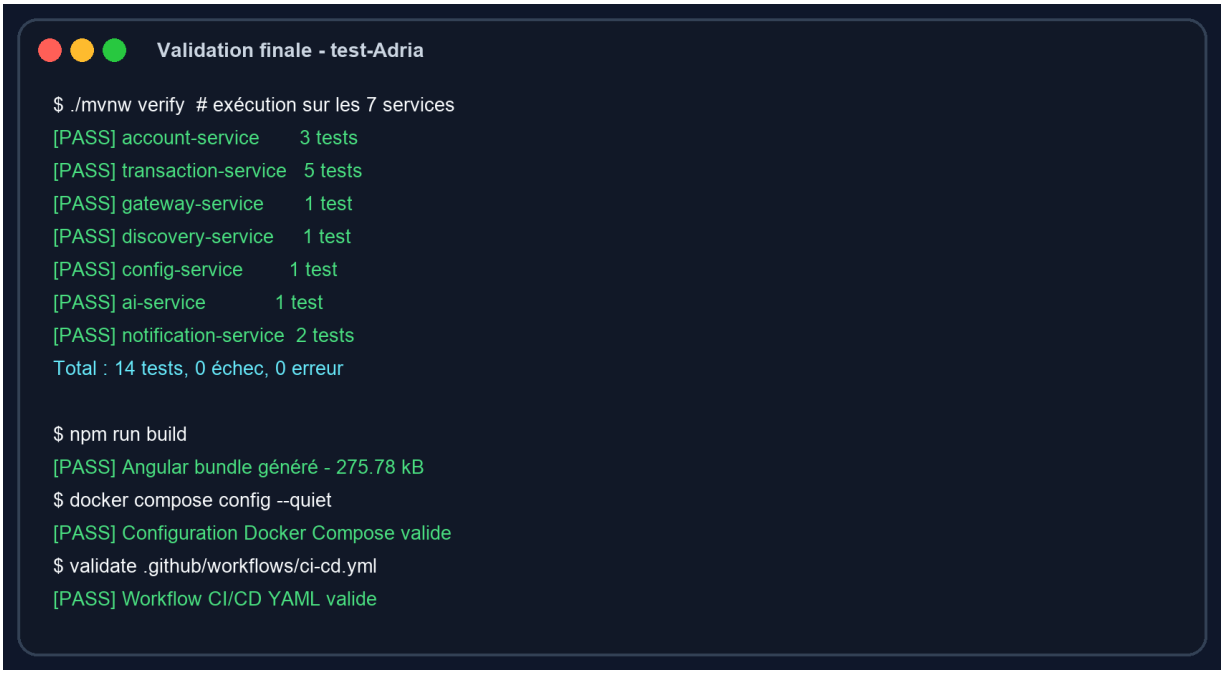


Figure 6 - Synthèse des validations exécutées localement.

13. Validation et conclusion

13.1 Résultats de validation

Composant	Tests	Résultat
account-service	3	Succès
transaction-service	5	Succès
gateway-service	1	Succès
discovery-service	1	Succès
config-service	1	Succès
ai-service	1	Succès
notification-service	2	Succès
Angular	Build production	Succès
Docker Compose	Validation de configuration	Succès
Workflow CI/CD	Validation YAML	Succès

13.2 Conclusion

La plateforme couvre les principaux objectifs de l'épreuve : architecture microservices, règles bancaires, connecteurs REST/GraphQL/MCP, résilience, configuration, découverte, agent IA, événements Kafka, client web, classification de fraude et chaîne DevOps. Les modules sont compilables, testés et versionnés séparément.

Améliorations recommandées

- Ajouter Keycloak et appliquer des rôles aux API sensibles.
- Implémenter le virement entre comptes avec idempotence et stratégie Saga/Outbox.
- Remplacer H2 par une base persistante en environnement d'intégration.
- Activer Prometheus, Grafana et OpenTelemetry dans Docker Compose.
- Utiliser une validation temporelle sur un dataset bancaire réel avant toute mise en production du modèle.